LABORATORIO 4

Sensores y Actuadores

Preparado por:
Luisdavid Colina V24766381
Samantha Ramírez V31307714

1. Hardware utilizado

Antes de entrar en las actividades conviene dejar constancia de dos diferencias respecto a lo que plantea la guía, porque explican los valores que aparecen en todas las capturas de este informe.

La primera es la placa. La guía indica un ATmega2560, pero la mesa de trabajo tenía disponible una Arduino UNO. Para lo que pide el laboratorio la sustitución es transparente.

La segunda es el sensor. El módulo disponible fue un DHT-22 en lugar del DHT-11. Ambos se manejan con la misma librería `DHT.h` y se diferencian solo en la constante de tipo, así que el cambio se resolvió con una línea de código.

2. Monitoreo de datos ambientales

2.1. Actividad 6.1: Circuito de conexión del sensor

Enunciado: Muestre el circuito de conexión del DHT-11 a la placa, utilice para ello Fritzing. (1 punto)

El montaje físico correspondiente quedó registrado en la Figura 1 realizada con Tinkercad.

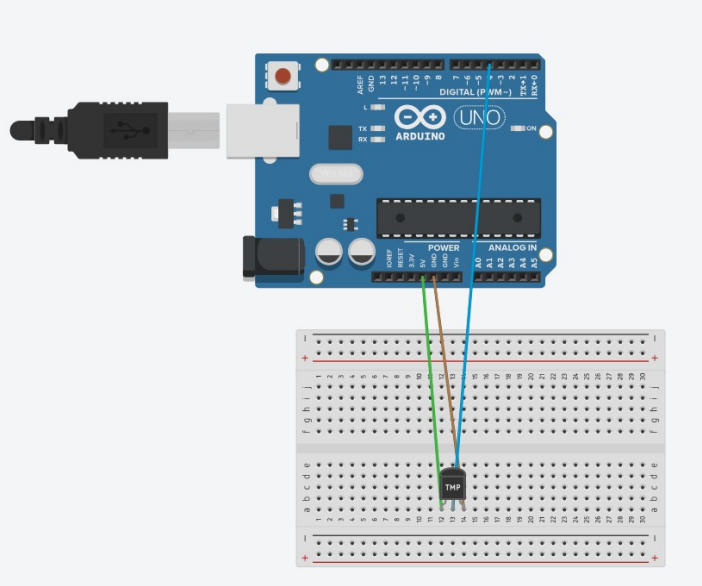


Figura 1: Circuito de conexión del sensor DHT-22 a la placa Arduino UNO.

2.2. Actividad 6.2: Lectura del sensor

Enunciado: Compile y suba el código 1 mostrado en los Anexos. Explique el funcionamiento general del código. Adjunte capturas de pantalla en el informe. (2 puntos)

El código del anexo se subió tal cual, con el único ajuste de las dos constantes que describen el hardware utilizado (Código 1). El resto del código quedó intacto.

Código 1: Ajuste de las constantes al hardware.

```
#define DHT_PIN     4       // Pin de datos del sensor
#define DHT_TYPE    DHT22   // Tipo de sensor disponible
```

```
/*
 * Laboratorio IoT - Actividad 1: Sensor DHT-11
 * Placa: ATmega2560 (Arduino Mega)
 */

#include "DHT.h"

// Constantes de configuracion
#define DHT_PIN       4        // Pin de datos del DHT-11
#define DHT_TYPE      DHT22    // Tipo de sensor

// Instancia del sensor DHT
DHT sensorDHT(DHT_PIN, DHT_TYPE);

// Variables para almacenar lecturas
float temperatura = 0.0;
float humedad = 0.0;

void setup() {
  Serial.begin(9600);
  while (!Serial);          // Esperar conexion (Mega compatible)

  Serial.println(F("========================================"));
  Serial.println(F("  LABORATORIO IoT - SENSOR DHT-11"));
  Serial.println(F("========================================"));
  Serial.println(F("Inicializando sensor..."));

  sensorDHT.begin();

  // Pequena pausa para estabilizacion
  delay(2000);
```

Figura 2: Código de lectura del sensor cargado en el IDE de Arduino.

El funcionamiento general es el siguiente. En `setup()` se abre el puerto serial a 9600 baudios, se inicializa el sensor con `sensorDHT.begin()` y se espera dos segundos antes de la primera lectura, porque el DHT necesita ese tiempo para estabilizar su circuito interno de medición.

El lazo `loop()` empieza con un `delay(2000)`. Esa pausa no es decorativa: la frecuencia de muestreo del sensor es de 1 Hz, de modo que pedirle lecturas más seguido devuelve el valor almacenado de la medición anterior en lugar de una lectura nueva.

Luego se llama a `readHumidity()` y `readTemperature()`, y el resultado se valida con `isnan()` antes de usarlo. Esa validación es necesaria porque la librería no lanza errores: cuando la trama de 40 bits llega corrupta o no llega, devuelve un valor no numérico. Sin esa comprobación el programa seguiría imprimiendo basura como si fueran mediciones válidas.

Con los datos ya validados se calcula el índice de calor mediante `computeHeatIndex()`, que combina temperatura y humedad en la temperatura aparente, y finalmente se imprime el bloque formateado junto con una clasificación por umbrales, que marca la temperatura como alta por encima de 30°C y la humedad como alta por encima de 70 %.

La Figura 3 muestra cuatro ciclos consecutivos del monitor serial. Las lecturas se mantuvieron alrededor de 26,0°C y 47 % de humedad relativa, ambas dentro del rango normal según los umbrales del propio código.

```
--------------------------------
Humedad Relativa:    47.70 %
Temperatura:         26.00 C  (78.80 F)
Indice de Calor:     25.90 C
Estado: TEMPERATURA NORMAL | HUMEDAD NORMAL
--------------------------------

--------------------------------
Humedad Relativa:    47.60 %
Temperatura:         26.00 C  (78.80 F)
Indice de Calor:     25.90 C
Estado: TEMPERATURA NORMAL | HUMEDAD NORMAL
--------------------------------

--------------------------------
Humedad Relativa:    47.60 %
Temperatura:         26.00 C  (78.80 F)
Indice de Calor:     25.90 C
Estado: TEMPERATURA NORMAL | HUMEDAD NORMAL
--------------------------------

--------------------------------
Humedad Relativa:    48.00 %
Temperatura:         26.00 C  (78.80 F)
Indice de Calor:     25.91 C
Estado: TEMPERATURA NORMAL | HUMEDAD NORMAL
--------------------------------
```

Figura 3: Salida del monitor serial con las lecturas del sensor.

2.3. Actividad 6.3: Lectura desde un script en Python

Enunciado: Investigue de qué manera un script en Python puede obtener los datos que el sensor DHT-11 escribe en el puerto serial. Programe en Python un ejemplo de uso que muestre los datos del sensor, debe instalar en Python la librería `pyserial`*. (2 puntos)*

Esta actividad no se realizó en el laboratorio, sin embargo se investigó lo siguiente: El microcontrolador no expone los datos de ninguna forma especial. Todo lo que hace es escribir texto en el puerto serial con `Serial.println()`, y ese puerto, del lado del PC, aparece como un archivo de dispositivo (`/dev/ttyUSB0` en el Linux Mint de la mesa de trabajo). Lo que aporta `pyserial` es la capa que configura la velocidad, la paridad y los tiempos de espera, y que entrega el flujo de bytes ya listo para leer [3].

3. Control de dispositivo de potencia

3.1. Actividad 6.5: Circuito de conexión del relé

Enunciado: Muestre el circuito de conexión del Relé a la placa, utilice para ello Fritzing. (1 punto)

El montaje físico correspondiente quedó registrado en la Figura 4 realizada con Tinkercad.

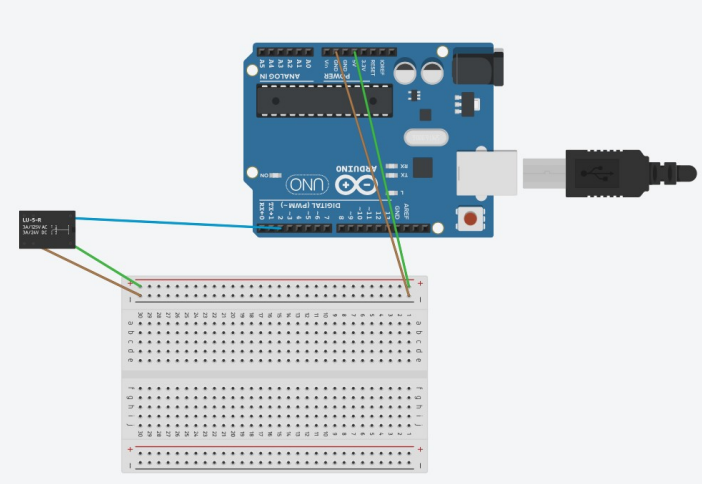


Figura 4: Circuito de conexión del relé a la placa Arduino UNO.

3.2. Actividad 6.6: Control básico del relé

Enunciado: Compile y suba el código 2 mostrado en los Anexos. Explique el funcionamiento general del código. Adjunte capturas de pantalla en el informe. (2 puntos)

El único ajuste al código del anexo fue mover el pin de control a digital 2, que es donde quedó cableada la entrada `IN` del módulo.

El código hace dos cosas a la vez dentro de `loop()`. Por un lado mantiene una secuencia automática de prueba que invierte el estado del relé cada cinco segundos, pero lo hace comparando `millis()` contra una marca de tiempo anterior en lugar de usar `delay()`. Esa distinción importa, porque `delay()` congelaría la placa durante esos cinco segundos y no podría atender nada más. Por otro lado revisa en cada vuelta si llegó un carácter por el puerto serial y, de ser así, lo despacha a `procesarComando()`.

El detalle que más confunde de este montaje es que el módulo es activo en bajo. Escribir `LOW` en el pin de control energiza la bobina y escribir `HIGH` la desenergiza, que es lo contrario de lo que sugiere la intuición. El sketch lo resuelve con el operador ternario `estadoRele ? LOW : HIGH`, y por eso el estado inicial seguro en `setup()` es `HIGH`.

Código 2: Conmutación del relé y atención de comandos en `loop()`.

```
if (tiempoActual - tiempoAnterior >= INTERVALO_PRUEBA) {
    tiempoAnterior = tiempoActual;

    estadoRele = !estadoRele;
    cicloPrueba++;

    digitalWrite(PIN_RELE, estadoRele ? LOW : HIGH); // Activo LOW
    digitalWrite(PIN_LED_IND, estadoRele ? HIGH : LOW);
}

if (Serial.available() > 0) {
    char comando = Serial.read();
    procesarComando(comando);
}
```

3.3. Actividad 6.7: Relé con estado persistente

Enunciado: Modifique el código para que el Relé mantenga el estado y solo se modifique cuando se le indique a través del puerto serial. (2 puntos)

La modificación consistió en eliminar por completo el bloque de la secuencia automática, que era lo único que cambiaba el actuador por su cuenta. Sin ese bloque el lazo queda reducido a atender comandos, y el relé conserva indefinidamente el último estado que se le indicó.

Aprovechando el cambio se centralizó la escritura al hardware en una función `aplicarEstado()`. En el sketch original cada rama del `switch` repetía las dos llamadas a `digitalWrite()` junto con la asignación a `estadoRele`, y ese patrón hace fácil que el estado lógico y el estado físico terminen desincronizados si alguna rama se edita y otra no.

Código 3: Lazo principal y escritura centralizada, en `src/rele_estado.ino`.

```
void loop() {
  // El lazo no toca el actuador, solo atiende comandos entrantes
  if (Serial.available() > 0) {
    char comando = Serial.read();
    procesarComando(comando);
  }
}

void aplicarEstado(bool encendido) {
  estadoRele = encendido;
  digitalWrite(PIN_RELE, encendido ? LOW : HIGH); // Activo LOW
  digitalWrite(PIN_LED_IND, encendido ? HIGH : LOW);
}
```

Se agregaron además dos casos que ignoran `'\r'` y `'\n'`. Sin ellos, el monitor serial configurado en modo *New Line* agrega el salto al final de cada envío y el sketch responde con el mensaje de comando no reconocido después de cada orden válida.

3.4. Actividad 6.8: Consulta y control del relé desde Python

Enunciado: Investigue de qué manera un script en Python podría leer el estado actual del Relé y adicionalmente enviar comandos a la placa para modificar el estado del actuador. Programe en Python un ejemplo de uso, debe instalar en Python la librería ***`pyserial`****. (2 puntos)*

Esta actividad no se realizó en el laboratorio, sin embargo se investigó lo siguiente: La clave está en que el puerto serial es bidireccional, de modo que el mismo enlace que se usa para leer sirve para escribir. El código ya define un protocolo mínimo de un carácter por comando, así que del lado de Python basta con escribir ese carácter y leer las líneas que la placa devuelva.

4. Automatización mediante sensor y actuador

4.1. Actividad 6.10: Circuito con sensor y actuador

Enunciado: Muestre el circuito de conexión que incluya al sensor y al actuador, utilice para ello Fritzing. (1 punto)

El montaje físico correspondiente quedó registrado en la Figura 5 realizada con Tinkercad.

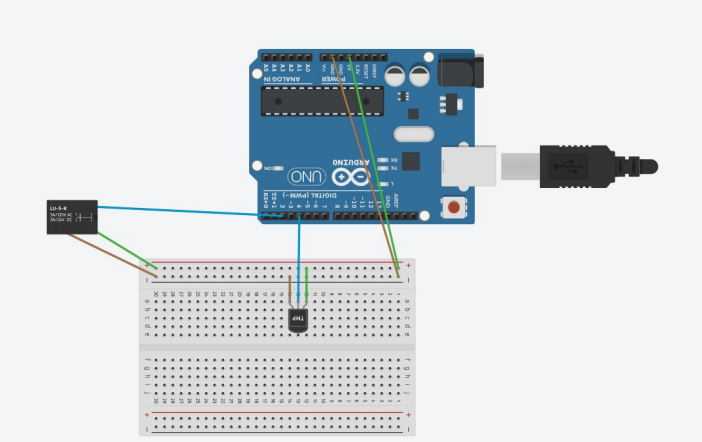


Figura 5: Circuito de conexión del DHT y relé a la placa Arduino UNO.

El montaje final integró los dos elementos sobre la misma placa. El sensor quedó en la protoboard con su línea de datos hacia el pin 4, y el módulo de relé recibió la señal de control en el pin 2, con la alimentación de ambos tomada de los rieles de 5 V y GND (Figuras 6 y 7).

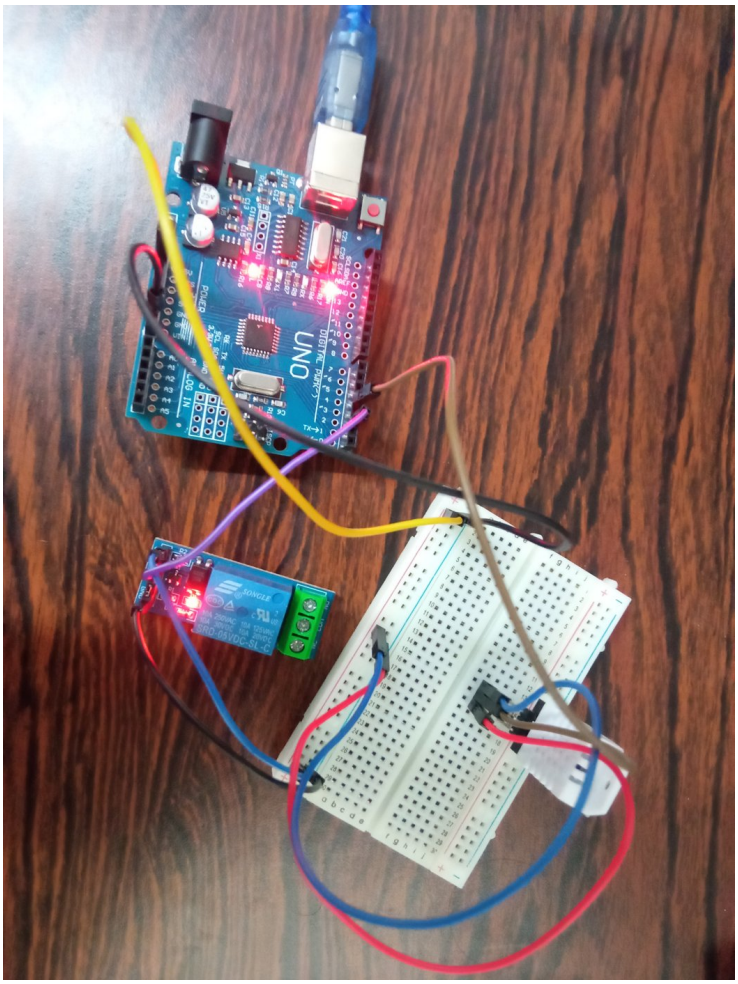


Figura 6: Montaje del sensor y el relé sobre la Arduino UNO.

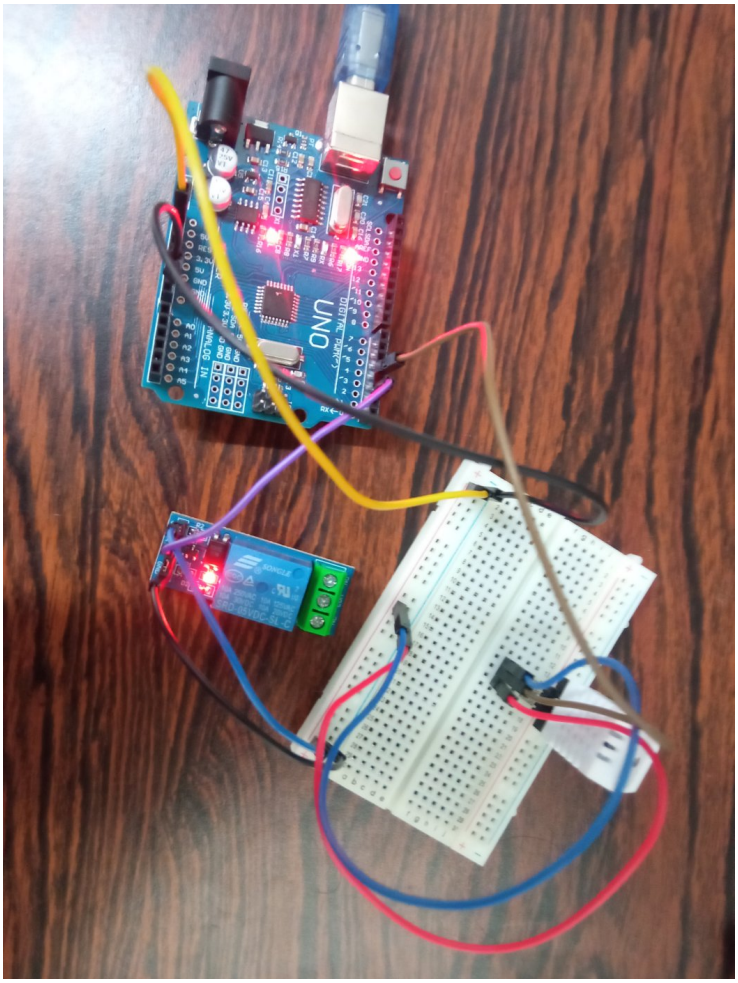


Figura 7: Segunda vista del montaje, con el cableado de control y alimentación.

4.2. Actividad 6.11: Automatización por umbrales

Enunciado: Programe, en el ATmega2560, el código para que el Relé mantenga el estado y solo se modifique cuando se le indique a través del puerto serial. (2 puntos)

El sketch resultante combina las dos mitades anteriores. Lee temperatura y humedad cada dos segundos, las compara contra dos umbrales configurables y acciona el relé cuando cualquiera de los dos se supera. Basta con que una de las dos variables se salga de rango para que el actuador se active (Código 4).

Código 4: Lógica de control por umbrales, en `src/dht11_rele.ino`.

```
float limiteTemperatura = 30.0;
float limiteHumedad = 60.0;

if (temperatura >= limiteTemperatura || humedad >= limiteHumedad) {
    digitalWrite(PIN_RELE, LOW); // Activa el rele
    Serial.println(F("-> ALERTA: Limite superado (Temp o Hum). Rele ENCENDIDO."));
} else {
    digitalWrite(PIN_RELE, HIGH); // Apaga el rele
    Serial.println(F("-> Condiciones normales. Rele APAGADO."));
}
```

```
1  #include "DHT.h"
2
3  // Definición de pines según tu configuración
4  #define DHT_PIN       4     // Pin de datos del sensor DHT22
5  #define DHT_TYPE      DHT22 // Tipo de sensor
6  #define PIN_RELE      2     // Pin de control del relé
7
8  // Instancia del sensor DHT
9  DHT sensorDHT(DHT_PIN, DHT_TYPE);
10
11 float temperatura = 0.0;
12 float humedad = 0.0;
13
14 // Límites ambientales para encender el relé (puedes ajustarlos a tu gusto)
15 float limiteTemperatura = 30.0;
16 float limiteHumedad = 60.0;     // Nuevo límite de humedad
17
18 void setup() {
19   Serial.begin(9600);
20
21   // Configurar el pin del relé como salida
```

Figura 8: Sketch integrado cargado en el IDE, con los umbrales configurados.

El umbral de humedad se ajustó durante la prueba de 70 % a 60 % para acercarlo a las condiciones del salón, donde las lecturas rondaban el 50 %. Aun así no se alcanzó a cruzar el umbral, de modo que la Figura 9 documenta la rama de condiciones normales con el relé apagado, que es el comportamiento correcto para esos valores.

```
--------------------------------
Temperatura: 26.10 C | Humedad: 55.50 %
-> Condiciones normales. Relé APAGADO.
--------------------------------
Temperatura: 26.10 C | Humedad: 52.70 %
-> Condiciones normales. Relé APAGADO.
--------------------------------
Temperatura: 26.20 C | Humedad: 51.30 %
-> Condiciones normales. Relé APAGADO.
--------------------------------
Temperatura: 26.10 C | Humedad: 50.60 %
-> Condiciones normales. Relé APAGADO.
--------------------------------
Temperatura: 26.10 C | Humedad: 50.00 %
-> Condiciones normales. Relé APAGADO.
--------------------------------
Temperatura: 26.20 C | Humedad: 50.10 %
-> Condiciones normales. Relé APAGADO.
--------------------------------
```

Figura 9: Telemetría del sistema integrado con el relé en reposo.

5. Conclusiones

El laboratorio dejó claro que trabajar con un sensor digital no se reduce a leer un valor. El DHT entrega una trama de 40 bits que puede llegar corrupta, y la librería señala esa falla devolviendo un valor no numérico en lugar de lanzar un error. La validación con `isnan()` no es una precaución opcional, es lo único que separa una medición real de un dato sin sentido.

Del lado del actuador, el punto que más costó asimilar fue la lógica invertida del módulo de relé. Un módulo activo en bajo se energiza escribiendo `LOW`, de modo que el estado seguro al arrancar es `HIGH`. Fijar ese estado en `setup()` evita que la carga quede conectada durante el arranque de la placa.

Separar el estado lógico del estado físico resultó ser la decisión más útil del laboratorio. Concentrar las llamadas a `digitalWrite()` en una sola función hace imposible que una rama del código cambie el actuador sin actualizar la variable que lo describe, que es justamente el error que después Python no podría detectar cuando consulta el estado por el puerto serial.

Finalmente, la automatización por umbrales mostró el esquema completo de un sistema IoT sencillo: el sensor entrega el dato, la lógica en el microcontrolador decide y el actuador ejecuta, todo sin intervención del PC. El PC queda solo como observador a través del puerto serial, que es exactamente la separación entre la lógica de interfaz y la ejecución en hardware que plantea la introducción de la guía.

Referencias

[1] Adafruit Industries, "DHT-sensor-library," *GitHub Repository*, 2012–2024. [En línea]. Disponible: `https://github.com/adafruit/DHT-sensor-library`. [Consulta: sep. 2026]

[2] Aosong Electronics, "DHT22 (AM2302) Digital-output relative humidity and temperature sensor," hoja de datos técnica, 2024. [En línea]. Disponible: `https://www.sparkfun.com/datasheets/Sensors/Temperature/DHT22.pdf`. [Consulta: sep. 2026]

[3] C. Liechti, "pySerial Documentation," 2001–2024. [En línea]. Disponible: `https://pyserial.readthedocs.io/en/latest/`. [Consulta: sep. 2026]

[4] Arduino, "Arduino Language Reference," *Arduino Documentation*, Arduino LLC, 2024. [En línea]. Disponible: `https://docs.arduino.cc/language-reference/`. [Consulta: sep. 2026]

[5] A. Russoniello, "Laboratorio: Sensores y Actuadores," guía de práctica de laboratorio, Laboratorio General (Internet de las Cosas), Escuela de Computación, Facultad de Ciencias, Universidad Central de Venezuela, Caracas, 2026.